

Curtin University – Department of Computing

Assignment Cover Sheet / Declaration of Originality

Complete this form if/as directed by your unit coordinator, lecturer or the assignment specification.

Last name:		Student ID:	
Other name(s):			
Unit name:		Unit ID:	
Lecturer / unit coordinator:		Tutor:	
Date of submission:		Which assignment?	(Leave blank if the unit has only one assignment.)

I declare that:

- The above information is complete and accurate.
- The work I am submitting is *entirely my own*, except where clearly indicated otherwise and correctly referenced.
- I have taken (and will continue to take) all reasonable steps to ensure my work is *not accessible* to any other students who may gain unfair advantage from it.
- I have *not previously submitted* this work for any other unit, whether at Curtin University or elsewhere, or for prior attempts at this unit, except where clearly indicated otherwise.

I understand that:

- Plagiarism and collusion are dishonest, and unfair to all other students.
- Detection of plagiarism and collusion may be done manually or by using tools (such as Turnitin).
- If I plagiarise or collude, I risk failing the unit with a grade of ANN ("Result Annulled due to Academic Misconduct"), which will remain permanently on my academic record. I also risk termination from my course and other penalties.
- Even with correct referencing, my submission will only be marked according to what I have done myself, specifically for this assessment. I cannot re-use the work of others, or my own previously submitted work, in order to fulfil the assessment requirements.
- It is my responsibility to ensure that my submission is complete, correct and not corrupted.

Signature: _____ *melissa* _____ Date of signature: _____

(By submitting this form, you indicate that you agree with all the above text.)

1. Introduction Summary

This program checks whether an input $n \times n$ matrix is a magic square using multithreading in C with POSIX threads (pthreads). A magic square is a matrix that contains all integers from 1 to n^2 exactly once and where the sum of each row, column and both main and secondary diagonals equals to the same magic constant. The magic constant is calculated using the formula, $n(n^2 + 1)/2$.

The solution uses concurrent threads to divide the validation tasks into separate threads for better efficiency. Four worker threads are created to independently perform row validation, column validation, main and secondary diagonals validation and uniqueness validation at the same time. Each thread independently processes its assigned task updates the shared validation results once done. A global score variable is implemented to track the overall validity of the magic square, where valid components increase the score while invalid components decreases it. A mutex lock was also implemented so two threads cannot change the score at the same time.

The main thread, implemented in `mmsv.c`, is responsible for reading the input file, allocating memory for the matrix, initialising shared variables, spawning the worker threads and waiting for all threads to complete execution. After all the threads are executed, the main thread will print the final report that displays the status of each row, column, diagonal and the uniqueness of the matrix, along with the final score and the final verdict on whether the matrix is a valid magic square or not.

2. Synchronization

This program uses multiple shared global variables, including the matrix, validation result arrays (e.g. `valid_row`, `valid_col`, `valid_diags` and `valid_unique`) and a global score variable. The score variable is accessed and modified by all worker threads, making it a critical shared resource that requires synchronisation. A mutex lock is used to protect all updates to the score. Each thread must acquire the mutex before increasing or decreasing the score and release it immediately after the update. This ensures that only one thread updates the score at a time, preventing inconsistent final score.

The row thread updates `valid_row` and the score, the column thread updates `valid_col` and the score, the diagonal thread updates `valid_diags` and the score and the uniqueness thread updates `valid_unique` and the score. These validation arrays do not require mutex protection because each one is written by only one specific thread. The main thread reads all validation results and the final score after all worker threads have completed their executions using `pthread_join` which ensures all shared data is updated before being printed.

3. Testing and Issues

The program was tested using several input inputs. Two valid test cases were created, `input_valid.txt` and `input_valid_1.txt`, with $n = 3$ and $n = 4$ respectively, where both matrices satisfied all the magic square conditions. The program correctly validated all rows, columns, both main and secondary diagonals and uniqueness and calculated the expected score with a valid result.

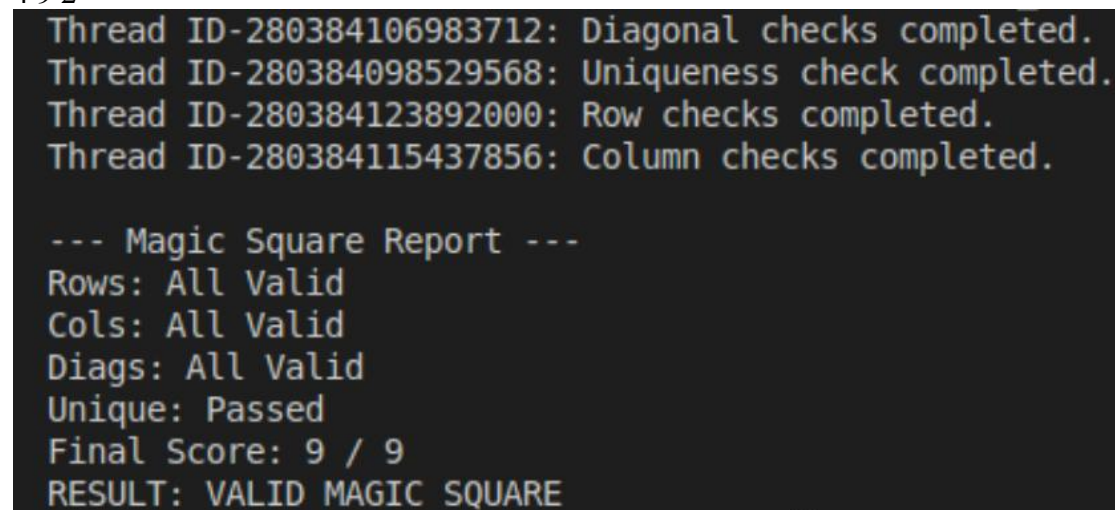
In addition, two invalid test cases were created, `input_dup_num.txt` and `input_diag_invalid.txt`. One test case contains a duplicate number which caused the uniqueness check to fail and the program correctly identified the issue. The other test case has a diagonal that did not match with the magic constant and the program successfully detected the incorrect diagonal sum. For both cases, the score was deducted accordingly and the final result was printed as invalid.

Based on these test runs, I believe the program behaves as expected and no issues were found during the testing.

4. Sample Input

a) Test 1 (Valid Magic Square)

```
n = 3
8 1 6
3 5 7
4 9 2
```

A screenshot of a terminal window showing the output of a program. The output consists of four lines of thread completion messages, followed by a section header '--- Magic Square Report ---' and several status lines. The status lines indicate that all rows, columns, and diagonals are valid, the uniqueness check passed, and the final score is 9 / 9. The final line of the report is 'RESULT: VALID MAGIC SQUARE'.

```
Thread ID-280384106983712: Diagonal checks completed.
Thread ID-280384098529568: Uniqueness check completed.
Thread ID-280384123892000: Row checks completed.
Thread ID-280384115437856: Column checks completed.

--- Magic Square Report ---
Rows: All Valid
Cols: All Valid
Diags: All Valid
Unique: Passed
Final Score: 9 / 9
RESULT: VALID MAGIC SQUARE
```

This shows that all rows, columns, both main and secondary diagonals produced the same sum and all numbers from 1 to 9 appeared exactly once. The program correctly validated all the components and printed the result as a magic valid square. Therefore, the output is correct.

b) Test 2 (Valid Magic Square)

n = 4
16 3 2 13
5 10 11 8
9 6 7 12
4 15 14 1

```
Thread ID-268231445246240: Diagonal checks completed.  
Thread ID-268231436792096: Uniqueness check completed.  
Thread ID-268231453700384: Column checks completed.  
Thread ID-268231462154528: Row checks completed.  
  
--- Magic Square Report ---  
Rows: All Valid  
Cols: All Valid  
Diags: All Valid  
Unique: Passed  
Final Score: 11 / 11  
RESULT: VALID MAGIC SQUARE
```

This shows that all rows, columns, both main and secondary diagonals produced the same sum and all numbers from 1 to 16 appeared exactly once. The program correctly validated all the components and printed the result as a magic valid square. Therefore, the output is correct.

c) Test 3 (Invalid Magic Square)

n = 3
8 1 6
3 5 8
4 9 2

```
Thread ID-250560298873120: Uniqueness check completed.  
Thread ID-250560307327264: Diagonal checks completed.  
Thread ID-250560315781408: Column checks completed.  
Thread ID-250560324235552: Row checks completed.  
  
--- Magic Square Report ---  
Rows: Row 2 Invalid  
Cols: Col 3 Invalid  
Diags: All Valid  
Unique: Failed (Duplicates found)  
Final Score: 3 / 9  
RESULT: INVALID MAGIC SQUARE
```

This shows that the program detected the duplicate number in the matrix which caused the uniqueness check to fail as well as the row and column validations. As a result, the score was decreased and the final result reported the matrix as an invalid magic square. Therefore, the output is correct.

d) Test 4 (Invalid Magic Square)

n = 3

8 1 6

3 5 7

4 9 6

```
Thread ID-276544798781728: Diagonal checks completed.  
Thread ID-276544790327584: Uniqueness check completed.  
Thread ID-276544815690016: Row checks completed.  
Thread ID-276544807235872: Column checks completed.  
  
--- Magic Square Report ---  
Rows: Row 3 Invalid  
Cols: Col 3 Invalid  
Diags: Main Diag Invalid  
Unique: Failed (Duplicates found)  
Final Score: 1 / 9  
RESULT: INVALID MAGIC SQUARE
```

This shows that the program detected the matrix had a diagonal that did not match the magic constant. The program correctly identified the incorrect diagonal sum, decreased the score and reported the matrix as an invalid magic square. It also detected that the row and column were invalid as well as the uniqueness check. Therefore, the output is correct.